

2: The merge function iterates through the two input lists and compares their elements one by one. Each element is moved into the result list exactly once, so the time complexity of the merge step is $O(n)$. In merge sort, the array is recursively divided into two halves until each subarray has only one element remaining. In each level of recursion, all elements are merged together. Since the array is divided into two halves each time, the number of recursion levels is $\log n$. In each level of recursion, the total workload of the merge operation is proportional to n . Therefore, the worst-case time complexity of this algorithm is $O(n \log n)$.

3: Give input vector

[8,42,25,3,3,2,27,3]

step1: divide

The array is first divided into two halves:

[8,42,25,3] [3,2,27,3]

divide each half again

[8,42] [25,3] [3,2] [27,3]

divide in to it self

[8] [42] [25] [3] [3] [2] [27] [3]

step2: Merge

Single-element arrays are merged in sorted order:

[8] + [42] = [8,42]

[25] + [3] = [3,25]

[3] + [2] = [2,3]

[27] + [3] = [3,27]

Doing same thing again:

[8,42] + [3,25] = [3,8,25,42]

[2,3] + [3,27] = [2,3,3,27]

Final merge:

[3,8,25,42] + [2,3,3,27] = [2,3,3,3,8,25,27,42]

The final sorted vector is:

[2,3,3,3,8,25,27,42]

4: Yes, the number of steps is consistent with my complexity analysis .

In my code, the array is divided into two halves each time in 'mergesort', so the number of recursion levels is about $\log n$.

At each level, the 'merge' function goes through all elements once, so the total work per level is $O(n)$.

Since there are $\log n$ levels and each level does $O(n)$ work, the total time complexity is $O(n \log n)$. For the example with 8 elements, there are 3 levels of merging, which matches the analysis.